Unit 5 - Database Connectivity with Java

Advanced Java (Pokhara University)

Unit 5: Database Connectivity with Java

- JDBC Architecture
- JDBC Driver Types and Configuration
- Managing Connection and Statements
- Result Sets and Exception Handling
- DDL and DML Operations
- SQL Injection and Prepared Statements
- Row Sets and Transactions
- SQL Escapes

Introduction

JDBC (Java Database Connectivity) is a Java-based API (Application Programming Interface) that provides a standard method for Java applications to interact with relational databases. It allows Java applications to establish database connections, send SQL queries to the database, retrieve data from the database, and update the database records. JDBC is a fundamental technology for building database-driven applications in Java. Some key features of JDBC are as follows.

1. **Database Connectivity:** JDBC provides a mechanism to establish a connection to a relational database management system (RDBMS). It supports a wide range of databases, including MySQL, Oracle, SQL Server, PostgreSQL, SQLite, and more.
2. **Platform Independence:** JDBC is designed to be platform-independent. This means that Java applications written using JDBC can run on different operating systems without modification. JDBC achieves this by using a driver manager that loads the appropriate database driver dynamically at runtime.
3. **SQL Query Execution:** With JDBC, SQL queries can be sent to the database. These queries can include SELECT statements for data retrieval or INSERT, UPDATE, DELETE statements for data manipulation.
4. **Result Processing:** JDBC allows processing the results of SQL queries. The results are typically returned as result sets, which can be iterated through to access the retrieved data.

5. **PreparedStatement:** JDBC provides a PreparedStatement interface that allows executing precompiled SQL statements. Prepared statements are more efficient and secure than regular statements because they can be reused with different parameter values.
6. **Transaction Management:** JDBC supports transaction management, allowing grouping multiple SQL statements into a single transaction. It can explicitly commit or rollback transactions, ensuring data integrity.
7. **Exception Handling:** JDBC methods can throw exceptions related to database connectivity, SQL syntax errors, and other issues. Proper exception handling is essential to manage these situations gracefully.
8. **Resource Management:** JDBC resources, such as connections, statements, and result sets, should be explicitly closed when they are no longer needed to release database resources and prevent resource leaks.
9. **Batch Processing:** JDBC allows performing batch processing, where multiple SQL statements are grouped together and executed in a single batch. This can improve performance when dealing with large datasets.
10. **Database Metadata:** Database metadata information can be retrieved using JDBC, which provides details about the database structure, such as tables, columns, indexes, and constraints.
11. **Driver Types:** JDBC drivers come in different types: Type-1 (JDBC-ODBC bridge), Type-2 (Native-API driver), Type-3 (Network Protocol driver), and Type-4 (Thin driver). Each type has its own characteristics and use cases.

Overall, JDBC is a critical technology for developing Java applications that require database connectivity. It simplifies the process of interacting with databases and provides a standardized way to work with different database systems. Whether building web applications, desktop applications, or server-side applications are built, JDBC enables to integrate database functionality seamlessly into Java programs.

JDBC, or Java Database Database Connectivity, is a Java API that is used to interact with databases, issue queries and commands, and process database result sets. JDBC and database drivers operate together to access spreadsheets and databases. The API classes and interfaces of JDBC allow an application to send a request to a particular database. JDBC API helps Java applications interact with ORACLE, MYSQL, and MSSQL databases.

Java programming language is used for developing enterprise applications. These applications are created with the purpose of solving real-life problems and need to interact with databases to store required data and perform operations on it. So, to interact with databases, there is a need for database connectivity, and for the same purpose, an Open Database Connectivity (ODBC) driver is used.

ODBC is an API by Microsoft and is used to interact with databases. ODBC can only be used by the Windows platform and can be used for any language such as C, C++, Ruby, Java, etc. On the other hand, JDBC is an API with interfaces and classes used to interact with databases. It is only used for Java languages and can be used for any platform. JDBC is highly suggested for Java applications because there are not any performance or platform dependency problems.

Components of JDBC

Following are the components of JDBC; these elements guide us in interacting with databases.

- JDBC API
- JDBC Test Suite
- JDBC Driver Manager
- JDBC-ODBC Bridge Drivers

JDBC API

It is a collection of various methods, classes, and interfaces for smooth communication with a database. It provides us with two different packages to connect with various databases.

```
java.sql.*;
javax.sql.*;
```

JDBC Test Suite

It is used to test operations that are being performed on a database with the help of JDBC drivers. It tests operations such as insertion, deletion, and updation.

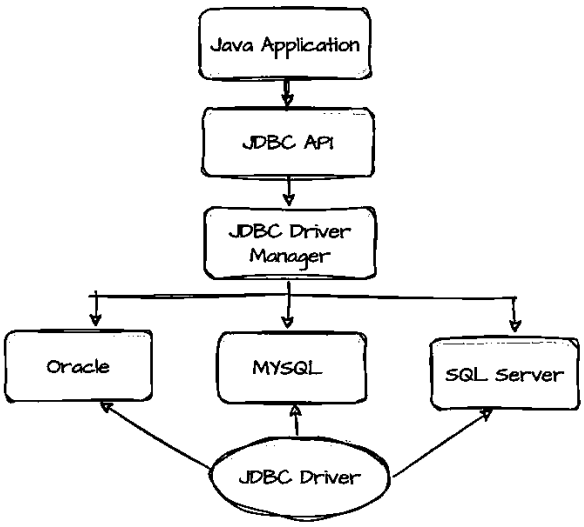
JDBC Driver Manager

A class in JDBC API that loads a database-specific driver in a Java application and creates a connection with a database is called JDBC Driver Manager. It makes a call to a particular database to process the user request.

JDBC-ODBC Bridge Drivers

JDBC-ODBC Bridge Drivers are used for the translation of JDBC methods into ODBC function calls. These are used to connect database drivers to the database. We require an ODBC connection for connection with the database even after using JDBC for Java Enterprise Applications. These are used to bridge the equal gap between JDBC and ODBC drivers. The bridge translates the object-oriented JDBC method call to the procedural ODBC function call. Then the sun.jdbc.odbc package is utilized.

Architecture of JDBC



As we can see from the above image, the important components of JDBC are:

- Application

  These are Java applications such as applets or servlet that communicates with databases

- JDBC API

  It is an API that is used to create Databases. It uses interfaces and classes to connect with databases. Some of the essential classes and interfaces defined in JDBC architecture in Java are the connection Interface and DriverManager class

- DriverManager

  This is used to create a connection between databases and Java applications. A connection is established between the Java application and data sources using the getConnection method of this Class

- Data Sources

  These are the databases that we can connect with the help of this API. These are the sources where the data is kept/stored and used by Java applications. JDBC API helps in connecting various databases such as MYSQL, MSSQL, PostgreSQL, Oracle, etc

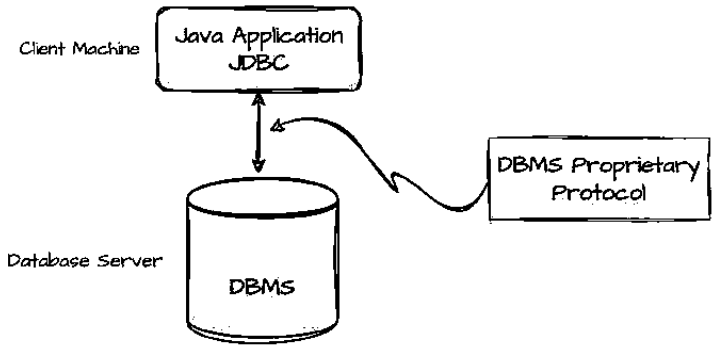
- JDBC Drivers

  These are used to connect with data sources. All databases, such as MSSQL, ORACLE, and MYSQL, have their drivers. We must load their specific drivers to connect with these databases. Java Class used for loading driver is Class. Class.forName() method is used to load drivers in JDBC architecture

Types of JDBC Architecture

Based on the processing models, the JDBC Architecture can be of two types. These models are the two-tier architecture and three-tier architecture.

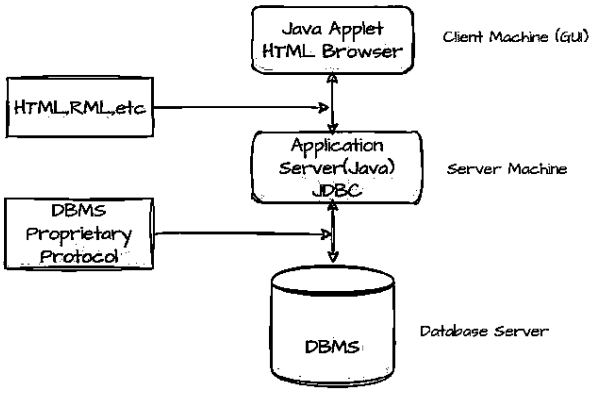
Two-Tier Architecture



- It is a basic model. In this Model, a Java application and Java applet communicate directly with the data source
- JDBC driver is used to create a connection between the data source and the application. Whenever an application is required to interact with a database, a query is executed directly on the data source, and the output of the queries is sent back to the user as a result

- This Model is also known as client/server configuration. Here, a user's machine acts as a client and the machine on which the database is placed acts as a server
- Here, the data source can be found on a different machine that is connected to the same network as the user.

Three-Tier Architecture



- This Model is a more secure and complex model of JDBC architecture in Java
- The user queries are sent to the middleware tier and then executed on the data source
- In this Model, the Java application is treated as one tier connected to the 3rd tier, i.e., data source using middle-tier services
- The results received on the database are again sent to the middle tier and then to the application/user

JDBC Driver Types and Configuration

JDBC (Java Database Connectivity) drivers are classified into four types based on their architecture and interaction with the database. Each type has its own advantages and limitations. The four types of JDBC drivers are explained as follows.

Type 1: JDBC-ODBC Bridge Driver (Bridge Driver)

The Type 1 driver, also known as the JDBC-ODBC Bridge, bridges JDBC to ODBC (Open Database Connectivity). It relies on the ODBC driver to connect to the database.

```
import java.sql.*;

public class Type1Example {
    public static void main(String[] args) {
        try {
            // Load the JDBC-ODBC Bridge driver
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

            // Establish a connection using ODBC
            String url = "jdbc:odbc:your_odbc_datasource";
            Connection  connection  =  DriverManager.getConnection(url,
"username", "password");

            // Perform database operations
            // ...

            // Close the connection
            connection.close();
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Type 2: Native-API Driver (Partially Java Driver)

The Type 2 driver uses a database-specific native library to communicate with the database. It translates JDBC calls into calls to the native database API. Examples of Type 2 drivers are specific to the database vendor. For instance, Oracle provides a Type 2 driver called OCI (Oracle Call Interface) driver.

```
import java.sql.*;

public class Type2Example {
    public static void main(String[] args) {
```

```
        try {
            // Load the Native-API driver
            Class.forName("com.vendor.NativeAPIDriver");

            // Establish a connection using the native API
            String url = "jdbc:vendor:your_database";
            Connection connection = DriverManager.getConnection(url,
"username", "password");

            // Perform database operations
            // ...

            // Close the connection
            connection.close();
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Type 3: Network Protocol Driver (Middleware Driver)

The Type 3 driver translates JDBC calls into an intermediate, DBMS-independent network protocol. The server-side middleware component converts the protocol into native database calls. This example demonstrates the use of a Type 3 driver with a middleware server. The actual code will vary based on the middleware product in use.

```
import java.sql.*;

public class Type3Example {
    public static void main(String[] args) {
        try {
            // Load the Type 3 driver (No specific driver class
needed)
```

```
            // Establish a connection using the network protocol
            String url = "jdbc:dbmiddleware:your_database";
            Connection connection = DriverManager.getConnection(url,
"username", "password");

            // Perform database operations
            // ...

            // Close the connection
            connection.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Type 4: Thin Driver (JDBC Net-Protocol Driver, Pure Java Driver)

The Type 4 driver communicates directly with the database using a vendor-specific protocol. It is a pure Java driver, making it platform-independent.

```
import java.sql.*;

public class Type4Example {
    public static void main(String[] args) {
        try {
            // Load the Thin driver
            Class.forName("oracle.jdbc.driver.OracleDriver");

            // Establish a connection using the Thin driver
            String url =
"jdbc:oracle:thin:@your_database_host:1521:your_sid";
```

```
            Connection connection = DriverManager.getConnection(url,
"username", "password");

            // Perform database operations
            // ...

            // Close the connection
            connection.close();
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
    }
}
```

It should be noted that the specific driver class and connection URL depend on the database vendor.
In the examples shown above, placeholders like your_odbc_datasource, com.vendor.NativeAPIDriver, jdbc:dbmiddleware:your_database, and jdbc:oracle:thin:@your_database_host:1521:your_sid are subjected to be replaced with actual values according to the database and driver configurations. Also, handle exceptions appropriately in a production environment.
The choice of JDBC driver type depends on factors such as performance, platform requirements, and database support. In modern applications, Type 4 drivers (Thin Drivers) are commonly used due to their platform independence and better performance compared to Type 1 and Type 2 drivers. The selection of the appropriate driver type is crucial for efficient and reliable database connectivity in Java applications.

Managing Connection and Statements

- JDBC allows establishing a connection to a relational database system. The java.sql.Connection interface represents a database connection.
- To connect to a database, we typically need to provide a database URL, username, and password.

```
import java.sql.Connection;
import java.sql.DriverManager;
```

```
Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb",
"username", "password");
```

Result Sets and Exception Handling

In Java programming, a ResultSet is an interface provided by the JDBC (Java Database Connectivity) API. It represents the result set of a database query and provides methods to traverse and access the data returned by the query. Exception handling is crucial when dealing with database operations as they involve potential errors such as connection failures, SQL syntax errors, and data retrieval issues. An overview of ResultSet and exception handling in the context of database operations are provided as follows

ResultSet

1. Creating a ResultSet

A ResultSet is typically obtained by executing a query on a Statement. A basic example is as follows.

```
try (Connection connection =
DriverManager.getConnection("jdbc:your_database_url", "username",
"password");
     Statement statement = connection.createStatement();
     ResultSet resultSet = statement.executeQuery("SELECT * FROM
your_table")) {

    // Process the ResultSet
    while (resultSet.next()) {
        // Retrieve data using resultSet.getXXX() methods
        int id = resultSet.getInt("id");
        String name = resultSet.getString("name");
        // Process data as needed
        System.out.println("ID: " + id + ", Name: " + name);
```

```
    }
} catch (SQLException e) {
    e.printStackTrace();
}
```

2. Traversing the ResultSet

The next() method is used to move the cursor to the next row in the result set. We can then use various getXXX() methods to retrieve data based on the data type of the column.

Exception Handling in JDBC

1. SQLException

JDBC operations can throw SQLException. It's a checked exception that needs to be handled.

2. Handling SQLException

We use try-catch blocks to handle SQLException. Common practices include logging the exception, rolling back transactions, and closing resources.

```
try (Connection connection = DriverManager.getConnection("jdbc:your_database_url", "username", "password");
     Statement statement = connection.createStatement();
     ResultSet resultSet = statement.executeQuery("SELECT * FROM your_table")) {

    while (resultSet.next()) {
        // Process the ResultSet
        int id = resultSet.getInt("id");
        String name = resultSet.getString("name");
        System.out.println("ID: " + id + ", Name: " + name);
    }

} catch (SQLException e) {
```

```
    e.printStackTrace();
    // Handle the exception: log, display an error message, etc.
}
```

3. Resource Closure in a try-with-resources Statement

In the example above, try-with-resources is used for the Connection, Statement, and ResultSet. This ensures that the resources are automatically closed, even if an exception occurs.

4. SQLException Handling Strategies

Strategies for handling SQLException may include:

- Logging the exception details.
- Displaying a user-friendly error message.
- Rolling back transactions if needed.
- Gracefully closing resources to avoid resource leaks.

5. SQLWarnings

Additionally, the SQLWarning class is used for warnings generated by the database. It can be retrieved using the getWarnings() method on the Connection or Statement.

```
try (Connection connection = DriverManager.getConnection("jdbc:your_database_url", "username", "password");
     Statement statement = connection.createStatement();
     ResultSet resultSet = statement.executeQuery("SELECT * FROM your_table")) {

    SQLWarning warning = connection.getWarnings();
    if (warning != null) {
        // Handle SQLWarnings
        while (warning != null) {
            System.out.println("SQLWarning: " + warning.getMessage());
            warning = warning.getNextWarning();
```

```
        }
    }

    // Rest of the code
    // ...

} catch (SQLException e) {
    e.printStackTrace();
    // Handle SQLException
}
```

It's important to handle exceptions appropriately in JDBC code to ensure robustness and reliability when interacting with databases. Proper exception handling helps in diagnosing and resolving issues during development, testing, and production use.

DDL and DML Operations

Creating Table (DDL Operation)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class CreateTableExample {
    public static void main(String[] args) {
        // JDBC URL for SQLite (Change this to your database URL if using a different DBMS)
        String jdbcUrl = "jdbc:sqlite:mydatabase.db";

        try {
            // Create a connection to the database
            Connection connection = DriverManager.getConnection(jdbcUrl);
```

```
            // Create a statement object
            Statement statement = connection.createStatement();

            // SQL statement to create a table
            String createTableSQL = "CREATE TABLE IF NOT EXISTS users
(" +
                    "id INTEGER PRIMARY KEY AUTOINCREMENT," +
                    "username TEXT NOT NULL," +
                    "email TEXT NOT NULL UNIQUE," +
                    "password TEXT NOT NULL" +
                    ")";

            // Execute the SQL statement to create the table
            statement.execute(createTableSQL);

            // Close the statement and connection
            statement.close();
            connection.close();

            System.out.println("Table 'users' created successfully.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Inserting Data (DML Operation)

To insert a record into a database using Java, we can continue from the previous example where we created a table. This example will show that how to insert a record into the "users" table using JDBC. Before running the following code, we have to make sure that we have set up a database and added the necessary JDBC driver to our project.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class InsertRecordExample {
    public static void main(String[] args) {
        // JDBC URL for SQLite (Change this to your database URL if using a different DBMS)
        String jdbcUrl = "jdbc:sqlite:mydatabase.db";

        try {
            // Create a connection to the database
            Connection connection = DriverManager.getConnection(jdbcUrl);

            // SQL statement to insert a record into the 'users' table
            String insertSQL = "INSERT INTO users (username, email, password) VALUES (?, ?, ?)";

            // Create a PreparedStatement to safely insert data with placeholders
            PreparedStatement preparedStatement = connection.prepareStatement(insertSQL);

            // Set values for the placeholders
            preparedStatement.setString(1, "Shirish Regmi");
            preparedStatement.setString(2, "contact@technopoets.com");
            preparedStatement.setString(3, "pwd123");

            // Execute the PreparedStatement to insert the record
            int rowsAffected = preparedStatement.executeUpdate();

            // Close the PreparedStatement and connection
```

```
            preparedStatement.close();
            connection.close();

            if (rowsAffected > 0) {
                System.out.println("Record inserted successfully.");
            } else {
                System.out.println("Record insertion failed.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In this example, we

- Define the JDBC URL for connecting to the SQLite database (jdbc:sqlite:mydatabase.db). It can be replaced with the appropriate URL for desired database system.
- Establish a connection to the database using DriverManager.getConnection().
- Define an SQL statement (insertSQL) to insert a record into the "users" table with placeholders for the values.
- Create a PreparedStatement to safely insert data with placeholders.
- Set values for the placeholders using setString().
- Execute the PreparedStatement using executeUpdate() to insert the record.
- Close the PreparedStatement and connection.
- Check the number of rows affected by the insertion and print a success or failure message.

It has to make sure to replace the values and column names with the actual data we want to insert into desired database. Additionally, handle exceptions and resource management as needed in a production environment.

Reading Data (DML Operation)

To read data from a database in a Java program, we can use JDBC (Java Database Connectivity), which provides a standard interface for connecting to relational databases. In the example as follows, we will

assume that we are working with a SQLite database, but we can adapt it for other databases with the appropriate JDBC driver.

Step 1: Set up the project

Firstly, it should be ensured that the JDBC driver for the desired database included in the project's classpath. For SQLite, we can download the SQLite JDBC driver (e.g., sqlite-jdbc-3.34.0.jar) from the SQLite JDBC website or use a build tool like Maven or Gradle to manage the dependencies.

Step 2: Write the Java program

In the following Java program, we can see that it connects to a SQLite database, retrieves data from a table, and prints it.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ReadDataFromDatabase {
    public static void main(String[] args) {
        String jdbcUrl = "jdbc:sqlite:mydatabase.db"; // Replace with your database URL

        try {
            // Create a connection to the database
            Connection connection = DriverManager.getConnection(jdbcUrl);

            // SQL query to retrieve data (change table name and columns as needed)
            String sqlQuery = "SELECT id, name, age FROM users";

            // Create a PreparedStatement to execute the query
```

```
            PreparedStatement            preparedStatement            =
connection.prepareStatement(sqlQuery);

            // Execute the query and get the result set
            ResultSet resultSet = preparedStatement.executeQuery();

            // Iterate through the result set and process the data
            while (resultSet.next()) {
                int id = resultSet.getInt("id");
                String name = resultSet.getString("name");
                int age = resultSet.getInt("age");

                System.out.println("ID: " + id + ", Name: " + name +
", Age: " + age);
            }

            // Close the result set, statement, and connection
            resultSet.close();
            preparedStatement.close();
            connection.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In this example,

- jdbcUrl specifies the JDBC URL for the database. We replace it with the appropriate URL for the desired database system.
- We establish a connection to the database using DriverManager.getConnection().
- The SQL query (sqlQuery) is defined to retrieve data from a hypothetical "users" table. It can be modified according to the desired database schema.
- We create a PreparedStatement to execute the query and retrieve the result set.

- We iterate through the result set, extract data, and print it to the console.
- Finally, we close the result set, prepared statement, and connection in a finally block to ensure proper resource management.

It should be make sure that we have the correct JDBC URL, SQL query, and database driver class based on our specific database system. Additionally, handle exceptions and resource management as needed in a production environment.

Updating Data (DML Operation)

To update a record in a database using Java, we can use JDBC (Java Database Connectivity). In this example, we update a record in the "users" table we created earlier. Before running the following code, we have set up a database and added the necessary JDBC driver to our project.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class UpdateRecordExample {
    public static void main(String[] args) {
        // JDBC URL for SQLite (Change this to your database URL if using a different DBMS)
        String jdbcUrl = "jdbc:sqlite:mydatabase.db";
        try {
            // Create a connection to the database
            Connection connection = DriverManager.getConnection(jdbcUrl);

            // SQL statement to update a record in the 'users' table
            String updateSQL = "UPDATE users SET password = ? WHERE username = ?";
```

```
            // Create a PreparedStatement to safely update data with
placeholders
            PreparedStatement            preparedStatement            =
connection.prepareStatement(updateSQL);

            // Set new password and username to update
            preparedStatement.setString(1, "new_password");
            preparedStatement.setString(2, "john_doe");

            // Execute the PreparedStatement to update the record
            int rowsAffected = preparedStatement.executeUpdate();

            // Close the PreparedStatement and connection
            preparedStatement.close();
            connection.close();

            if (rowsAffected > 0) {
                System.out.println("Record updated successfully.");
            } else {
                System.out.println("No records were updated.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In this example, we

- Define the JDBC URL for connecting to the SQLite database (jdbc:sqlite:mydatabase.db). it can be replaced this with the appropriate URL for desired database system.
- Establish a connection to the database using DriverManager.getConnection().
- Define an SQL statement (updateSQL) to update a record in the "users" table with placeholders for the values to be updated.
- Create a PreparedStatement to safely update data with placeholders.

- Set the new password and username to update using setString().
- Execute the PreparedStatement using executeUpdate() to update the record.
- Close the PreparedStatement and connection.
- Check the number of rows affected by the update and print a success or failure message.

It has to make sure to replace the values and column names with the actual data we want to update in the desired database. Additionally, handle exceptions and resource management as needed in a production environment.

Deleting Record (DML Operation)

To delete a record from a database using Java, we can use JDBC (Java Database Connectivity). In this example, we will see how to delete a record from the "users" table we created earlier. Before running the following code, we have to set up a database and add the necessary JDBC driver to the project.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class DeleteRecordExample {
    public static void main(String[] args) {
        // JDBC URL for SQLite (Change this to your database URL if
using a different DBMS)
        String jdbcUrl = "jdbc:sqlite:mydatabase.db";

        try {
            // Create a connection to the database
            Connection                  connection                  =
DriverManager.getConnection(jdbcUrl);

            // SQL statement to delete a record from the 'users' table
            String deleteSQL = "DELETE FROM users WHERE username = ?";
```

```
            // Create a PreparedStatement to safely delete data with
placeholders
            PreparedStatement           preparedStatement           =
connection.prepareStatement(deleteSQL);

            // Set the username to delete
            preparedStatement.setString(1, "john_doe");

            // Execute the PreparedStatement to delete the record
            int rowsAffected = preparedStatement.executeUpdate();

            // Close the PreparedStatement and connection
            preparedStatement.close();
            connection.close();

            if (rowsAffected > 0) {
                System.out.println("Record deleted successfully.");
            } else {
                System.out.println("No records were deleted.");
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In this example, we

- Define the JDBC URL for connecting to the SQLite database (jdbc:sqlite:mydatabase.db). it can be replaced this with the appropriate URL for desired database system.
- Establish a connection to the database using DriverManager.getConnection().
- Define an SQL statement (deleteSQL) to delete a record from the "users" table with a placeholder for the value to be deleted.
- Create a PreparedStatement to safely delete data with a placeholder.

- Set the username to delete using setString().
- Execute the PreparedStatement using executeUpdate() to delete the record.
- Close the PreparedStatement and connection.
- Check the number of rows affected by the delete operation and print a success or failure message.

It has to make sure to replace the values and column names with the actual data we want to delete in our database. Additionally, handle exceptions and resource management as needed in a production environment.

SQL Injection and Prepared Statements

SQL injection is a security vulnerability that occurs when an attacker is able to manipulate or inject malicious SQL code into a query, typically through user input. This can lead to unauthorized access, data manipulation, and other security breaches. Prepared Statements are a way to mitigate SQL injection risks by using parameterized queries. An explanation of SQL injection and how to use Prepared Statements in Java to prevent it is as follows,

In SQL injection attacks, attackers attempt to insert malicious SQL code into input fields or parameters. For example, consider a simple login query as,

```
SELECT * FROM users WHERE username = 'inputUsername' AND password = 'inputPassword';
```

If an attacker provides the following input for the username field,

```
' OR '1'='1' --
```

The resulting query becomes

```
SELECT * FROM users WHERE username = '' OR '1'='1' --' AND password = 'inputPassword';
```

This query will likely return all users in the database, as '1'='1' is always true.

Prepared Statements to Prevent SQL Injection

Prepared Statements help prevent SQL injection by separating SQL code from user input. Instead of directly embedding user input into the SQL query, placeholders are used, and the values are later bound to these placeholders. An example using a PreparedStatement in Java is as follows.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class LoginExample {
    public static void main(String[] args) {
        // JDBC URL, username, and password of your database server
        String url = "jdbc:mysql://localhost:3306/your_database";
        String user = "your_username";
        String password = "your_password";

        // User input (replace this with actual user input)
        String inputUsername = "john_doe";
        String inputPassword = "user_password";

        // SQL query using a prepared statement
        String sqlQuery = "SELECT * FROM users WHERE username = ? AND
password = ?";

        try (Connection connection = DriverManager.getConnection(url,
user, password);
             PreparedStatement preparedStatement =
connection.prepareStatement(sqlQuery)) {

            // Set values for placeholders
            preparedStatement.setString(1, inputUsername);
```

```
            preparedStatement.setString(2, inputPassword);

            // Execute the query
            ResultSet resultSet = preparedStatement.executeQuery();

            // Process the result set or check if user exists
            if (resultSet.next()) {
                System.out.println("Login successful!");
            } else {
                System.out.println("Invalid username or password.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In this example, the input values are set using setString() on the PreparedStatement, and they replace the placeholders (?). This ensures that user input is treated as data and not executable SQL code.
By using Prepared Statements, you significantly reduce the risk of SQL injection, as the database treats user input as data rather than executable code. Always validate and sanitize user input, and avoid building SQL queries by concatenating strings with user input.

Row Sets and Transactions

RowSet

A RowSet is a Java API that extends the functionality of ResultSet. It provides a more flexible and easier-to-use interface for working with database rows. RowSet implementations can be disconnected from the database, making them suitable for scenarios where a continuous database connection is not required.

1. Types of RowSets

There are several types of RowSets, including JdbcRowSet, CachedRowSet, WebRowSet, JoinRowSet, and FilteredRowSet. Each has its specific use cases.

2. Disconnected Operation

One of the advantages of RowSets is that they can operate in a disconnected mode. This means that after retrieving data from the database, the connection can be closed, and the RowSet can continue to work with the data locally.

3. Ease of Use

RowSets provide a simpler and more intuitive API compared to traditional ResultSet, making it easier to work with database rows.

For example,

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import javax.sql.rowset.CachedRowSet;
import javax.sql.rowset.RowSetFactory;
import javax.sql.rowset.RowSetProvider;

public class RowSetExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/your_database";
        String user = "your_username";
        String password = "your_password";

        try {
            // Create a RowSetFactory
            RowSetFactory rowSetFactory = RowSetProvider.newFactory();

            // Create a CachedRowSet
```

```
            CachedRowSet rowSet = rowSetFactory.createCachedRowSet();

            // Set connection parameters
            rowSet.setUrl(url);
            rowSet.setUsername(user);
            rowSet.setPassword(password);

            // Set SQL query
            rowSet.setCommand("SELECT * FROM your_table");

            // Execute the query and populate the RowSet
            rowSet.execute();

            // Process the data
            while (rowSet.next()) {
                // Access column data
                int id = rowSet.getInt("id");
                String name = rowSet.getString("name");
                // Process data as needed
                System.out.println("ID: " + id + ", Name: " + name);
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Transactions in Java

Transactions in Java, specifically in the context of JDBC, are used to group multiple database operations into a single atomic unit. A transaction ensures that either all the operations within the unit are completed successfully (commit), or none of them are executed (rollback) in case of an error.

1. Transaction Management

Transactions are managed through the Connection object in JDBC. The commit() method is used to apply changes, and the rollback() method is used to discard changes.

2. Auto-commit Mode

By default, a Connection is in auto-commit mode, where each SQL statement is treated as a separate transaction. You can disable auto-commit mode using the setAutoCommit(false) method.

3. Commit and Rollback

Once a set of operations is completed successfully, you can commit the changes using the commit() method. If an error occurs or if you want to discard the changes, you can use the rollback() method.

For example,

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class TransactionExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/your_database";
        String user = "your_username";
        String password = "your_password";

        try (Connection connection = DriverManager.getConnection(url,
user, password);
             Statement statement = connection.createStatement()) {

            // Disable auto-commit mode
            connection.setAutoCommit(false);

            try {
                // Perform multiple SQL operations
```

```
                statement.executeUpdate("INSERT     INTO     your_table
(column1, column2) VALUES (value1, value2)");
                statement.executeUpdate("UPDATE your_table SET column1
= new_value WHERE condition");
                statement.executeUpdate("DELETE  FROM  your_table  WHERE
condition");

                // Commit the transaction if all operations succeed
                connection.commit();
                System.out.println("Transaction                committed
successfully.");

            } catch (SQLException e) {
                // Rollback the transaction in case of an error
                connection.rollback();
                System.out.println("Transaction rolled back due to an
error.");
                e.printStackTrace();
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In this example, the setAutoCommit(false) method is used to disable auto-commit mode. The operations are grouped within a transaction, and the commit() method is called if all operations succeed. If an error occurs, the rollback() method is called to discard the changes. This ensures the atomicity of the transaction.

SQL Escapes

SQL escapes in Java refer to the process of properly handling special characters or reserved keywords in SQL queries to avoid SQL injection vulnerabilities. SQL injection occurs when untrusted input is directly concatenated into SQL queries, allowing attackers to manipulate the queries and potentially gain unauthorized access or manipulate data.

In Java, the PreparedStatement class is commonly used to execute SQL queries with proper handling of escapes. PreparedStatement supports parameterized queries, allowing you to set parameters without directly concatenating user input into the SQL query string. An example of using PreparedStatement to prevent SQL injection is as follows.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class PreparedStatementExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/your_database";
        String user = "your_username";
        String password = "your_password";

        String userInput = "john_doe";   // Replace this with actual
user input

        try (Connection connection = DriverManager.getConnection(url,
user, password)) {

            // Using PreparedStatement to handle escapes
            String sqlQuery = "SELECT * FROM users WHERE username =
?";
```

```
            try (PreparedStatement preparedStatement =
connection.prepareStatement(sqlQuery)) {

                // Set user input as a parameter
                preparedStatement.setString(1, userInput);

                // Execute the query
                ResultSet resultSet =
preparedStatement.executeQuery();

                // Process the result set or check if the user exists
                while (resultSet.next()) {
                    System.out.println("User found: " +
resultSet.getString("username"));
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

In this example, the PreparedStatement is used with a parameterized query ("SELECT * FROM users WHERE username = ?"). The user input is set as a parameter using the setString(1, userInput) method, which ensures that the input is properly escaped and prevents SQL injection.

The following points have to be brought into considerations.

- Use PreparedStatement to handle parameterized queries.
- Set parameters using specific setter methods (setString, setInt, etc.) to ensure proper escaping.
- Avoid concatenating user input directly into SQL queries.

By following these practices, the risk of SQL injection vulnerabilities in a Java application is reduced.